SProj 2026-27 | Group 4 "Height Boosters" | Supervisor: Dr. Ali Khawaja

LUMS Process Orchestrator

How our force-enrollment agent works, explained in plain language

This document walks through everything built so far (milestones M0 to M8): the problem, the ideas behind deep agents, how the system is put together, what happens step by step when a student asks for force enrollment, how safety is guaranteed, how it was tested, and what we learned along the way.

9	159	11	2,036
milestones built (M0-M8)	automated tests, no network	live tests on the real model	lines of application code

17 October 2026 | Repository state: commit 0058fb0 | All data is synthetic; no real student data or LUMS systems are used.

Contents

1. The problem and what we set out to build

2. The ideas behind it, in plain words

3. How the system fits together

4. One request from start to finish

5. The life of a case

6. The rules and the test students

7. Approvals: how a write is made safe

8. Memory: what the agent remembers, and where

9. Pausing for days and surviving a restart

10. Long conversations: compression and a cost cap

11. How we built it, milestone by milestone

12. How we know it works

13. What surprised us

14. deepagents or plain LangGraph?

15. Open questions and next steps

16. Glossary

1. The problem and what we set out to build

At LUMS, academic requests such as force enrollment travel through Zambeel, email and office visits. Nobody has one view of where a request stands, approvals happen in email threads, and students chase people for updates.

Our project puts a natural-language assistant in front of Zambeel (LUMS's Oracle PeopleSoft system). A student describes what they want in plain English. The assistant checks the rules, asks the student to confirm, asks the right instructor to approve, carries out only the approved action, and keeps a complete record of every step.

This repository is the **orchestrator**: the AI agent at the centre, built for one workflow end to end, **force enrollment** (getting into a section that is already full, with the instructor's approval). It runs against a **mock Zambeel**: a small database with made-up students and courses, because we do not have access to the real system yet.

What the build had to prove

Objective (from our BRD)	What it means here
O-01 End-to-end workflow	A request can go from the student's first message to a completed enrollment, with checks, confirmation, approval and notification.
O-02 Zero unauthorized writes	No enrollment is ever written without a valid instructor approval, whatever the AI model says or does.
O-03 Complete audit record	Every change to a case is recorded: who, what, from which state to which, and when.

> **In one sentence:** the student asks, the system checks the rules, the student confirms, the instructor approves, and only then is the enrollment written; every step is recorded.

The build also had a learning goal set by our supervisor: to exercise the main ideas of **deep agents** on a real workflow, so the team can explain each mechanism precisely. Section 2 explains those ideas.

2. The ideas behind it, in plain words

What an AI agent is

A language model on its own only writes text. An **agent** is a model given **tools** (functions it may call, such as "check my eligibility") and run in a loop: it reads the conversation, decides whether to call a tool, reads the result, and repeats until it can answer. The model decides what to do next; the tools are ordinary code that we write and control.

The three layers we build on



Each layer is built on the one below it. Only the top layer is our own code.

Figure 1. The software stack. LangChain, LangGraph and deepagents are open-source libraries from LangChain.

LangChain gives the basic parts: a common way to talk to any model and to define tools. **LangGraph** runs an agent as a series of steps and, crucially, can **save its exact state** after every step, pause, and continue later. **deepagents** is a ready-made agent built on LangGraph that bundles planning, helper agents, files and memory, so we did not have to wire those ourselves.

The deep-agent building blocks we used

Idea	Plain explanation	Where we use it
Planning (write_todos)	The agent writes a short to-do list before acting. It runs nothing; it keeps the model on track as the conversation grows.	Every force-enrollment request starts with a plan.
Skills	A rules file the agent opens only when it needs it. Only its name and a one-line description sit in the agent's instructions.	The force-enrollment rules FE-1 to FE-8 (Section 6).
Sub-agent (task tool)	A helper agent with its own clean context. It does one job and hands back only a short result, so the main conversation stays small.	The eligibility checker returns a structured verdict.
Human-in-the-loop pause (interrupt_on)	Before certain tools run, the agent stops and waits for a person to approve or reject.	Student confirmation and instructor approval.
Checkpointer (short-term memory)	Saves the whole state of one conversation after every step, in a file.	A paused case can wait for days and survive a restart.

Idea	Plain explanation	Where we use it
Store (long-term memory)	A small database shared across conversations, organised by namespace.	Per-student notes about past cases.
Runtime context	Facts the program (not the model) passes in on every run, such as who the student is.	Tools always act for the signed-in student only.
Structured output	The model must return data in a fixed shape (a schema), not free text.	The eligibility verdict.
Context compression	When a conversation gets long, older parts are summarized; huge tool results are saved to files.	Shown in a demo (Section 10).
Middleware	Small hooks that run around each model or tool call.	Safety guards and a cap on model calls.

The safety idea that shapes everything

An AI model can be wrong, can be talked into things, and behaves slightly differently each run. So we never rely on instructions to the model for anything that matters. Every consequential rule is **deterministic code** that runs no matter what the model says:

- **Approval token:** a digitally signed, single-use permission slip that only the instructor's script can create (Section 7).
- **State machine:** a case can only move along allowed paths; a rejected case can never be executed.
- **Idempotency:** repeating an enrollment request with the same case id can never enroll twice.
- **Audit log:** an append-only record of every change; the database itself refuses edits and deletions.
- **Read-only rules:** the agent cannot rewrite its own rules, skills or memory notes.

> **The principle:** the model decides *when* to ask for something; our code decides *whether* it may happen.

3. How the system fits together

Two people use the system, each through their own small program, and both work on the same cases through the same agent.

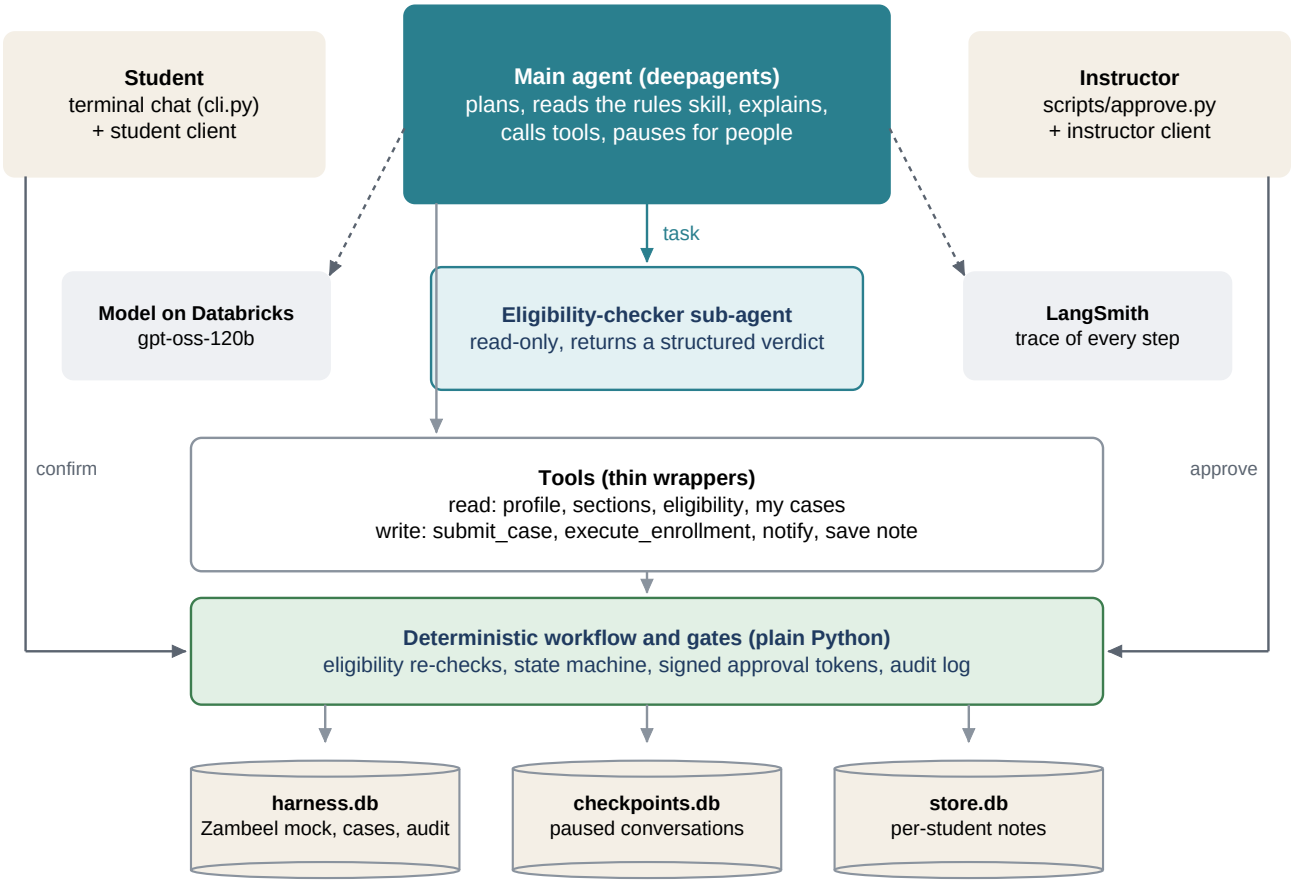


Figure 2. The parts of the system. Dashed lines: the agent calls the model and every step is traced.

Part	What it does
Student chat (cli.py)	A terminal chat. Each session is one conversation thread whose id is also the id of the case it may create (for example FE-45c1d9b5). It also answers the agent's confirmation pauses.
Instructor script (approve.py)	Lists cases waiting for an instructor, records an approve or reject decision, and wakes the paused conversation up again.
Main agent	Talks to the student, plans, reads the rules, delegates eligibility checks, calls tools, and pauses for people.
Eligibility-checker sub-agent	Runs the eligibility check in its own context and returns only a verdict. It cannot write anything.
Tools	Thin wrappers. They always take the student from the runtime context and the case from the thread, never from the model.
Workflow and gates	Plain Python that enforces every rule: re-checks, state changes, token checks, enrollment, audit.
Three database files	harness.db (mock Zambeel, cases, approvals, audit, notices), checkpoints.db (paused conversations), store.db (per-student notes).
LangSmith	Records a trace of every model call and tool call, so each step can be inspected afterwards.

4. One request from start to finish

Here is what happens when student S1 asks for a place in the full section CS412-S1. The steps run in two separate programs: the student's chat (steps 1-7) and, later, the instructor's script (steps 8-11).

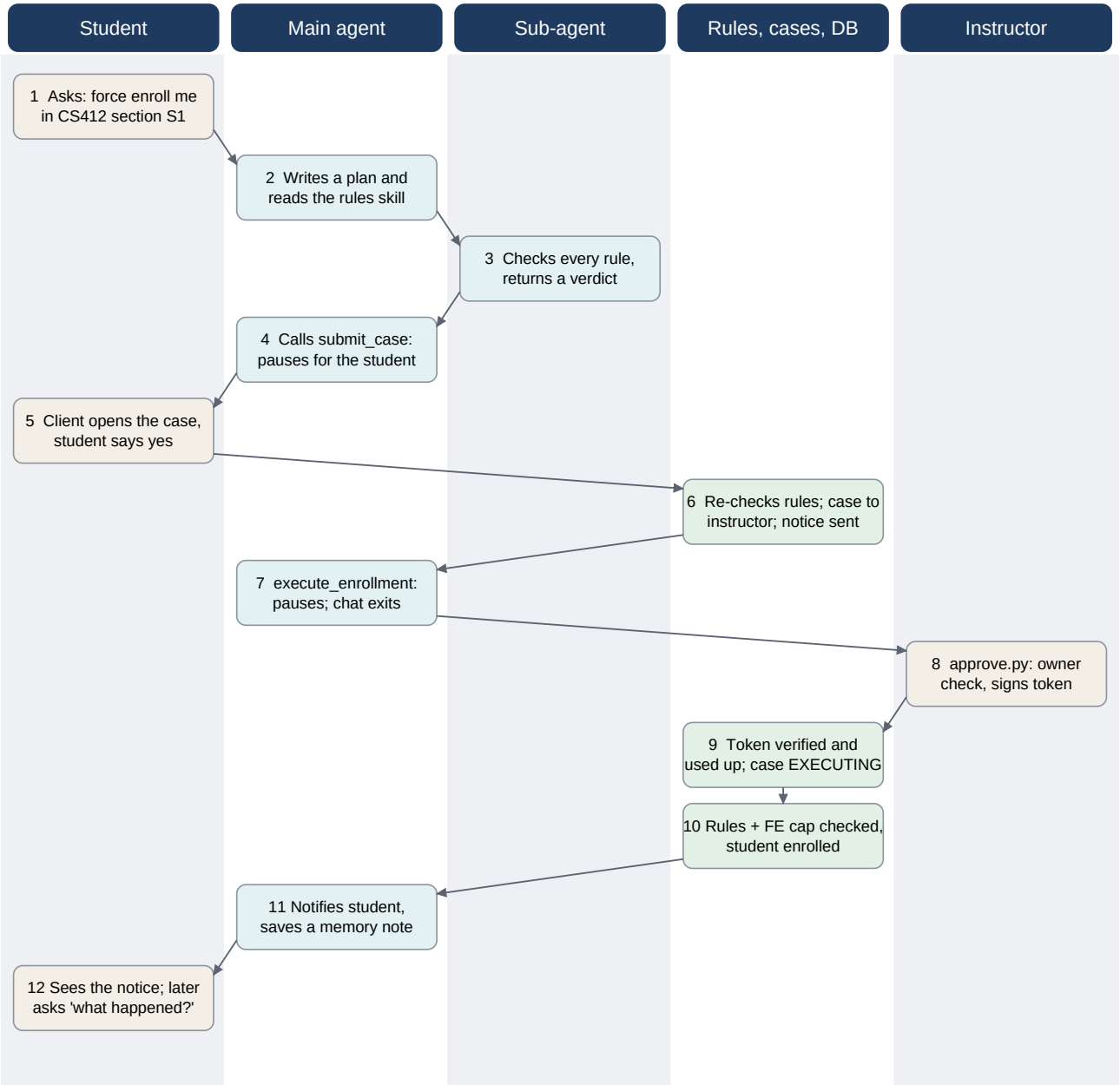


Figure 3. The force-enrollment journey across the people and parts involved. Read top to bottom.

What happens at each step

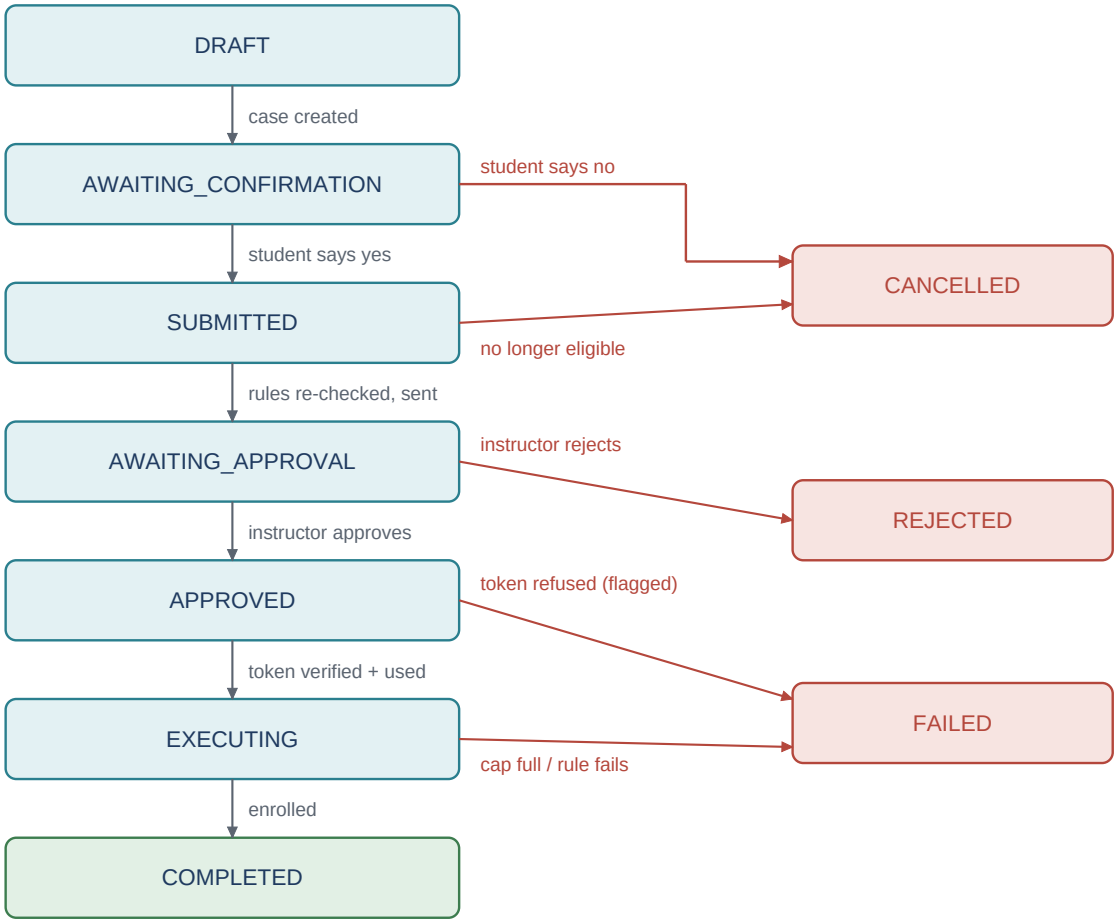
- **1-2.** The student writes in plain English. The agent writes a short plan and opens the force-enrollment rules file.
- **3.** The agent hands the eligibility check to the sub-agent. It checks prerequisites, timetable clashes, the credit limit, probation, section restrictions and whether the instructor accepts force enrollment, then returns a verdict such as: can submit, with a note that section S2 has open seats.

- **4-5.** The agent calls submit_case and the system pauses. The student's program re-checks the rules itself (it never trusts the verdict), creates the case, and asks the student to confirm. The student's yes or no is recorded.
- **6.** submit_case re-checks the rules once more, moves the case to the instructor and sends the instructor a notice.
- **7.** The agent calls execute_enrollment, which pauses until the instructor decides. The chat program can now close.
- **8.** Later, the instructor runs approve.py. It checks that this instructor owns the section, records the decision and, on approval, creates a signed one-time token.
- **9-10.** The paused conversation is woken up. execute_enrollment verifies the token, marks it used, re-checks the student's rules (things may have changed), checks there is still a force-enrollment slot, and writes the enrollment.
- **11-12.** The student gets a notice, the agent saves a one-line note to the student's long-term memory, and in any later session the agent can answer "what happened with my last request?"

> If the instructor rejects, the tool never runs, the case ends REJECTED and the student is told who decided and why. If the student says no at step 5, the case ends CANCELLED and the instructor never hears about it.

5. The life of a case

Every case moves through a fixed set of states. One function in the code is the only way to change a case's state, and it checks every move against this diagram and records it in the audit log in the same database transaction.



Green: success. Red: terminal states that never write an enrollment. A rejected case can never reach EXECUTING.

Figure 4. The case state machine. The audit log holds one row for every arrow a case takes.

Two paths were added during the build and are now part of the design: SUBMITTED to CANCELLED (the student is no longer eligible when the rules are re-checked) and APPROVED to FAILED (the approval token could not be verified, so the case is flagged for a person to review).

6. The rules and the test students

The rules are **placeholders written by the team**, not official LUMS policy, so every check could be exercised in tests. Each rule has a label that the agent must cite in its answers, so every explanation can be traced to a rule (our stand-in for the RAG citation requirement).

Label	Rule (placeholder v0.1)	Codes the checker reports
FE-1	Force enrollment is for full sections; with open seats, register normally	SEATS_AVAILABLE, OTHER_SECTION_OPEN (advisory)
FE-2	Prerequisites must be completed; in progress does not count	PREREQ_MISSING, PREREQ_IN_PROGRESS
FE-3	No timetable clash with current courses	CLASH
FE-4	At most 20 credits including the new course	CREDIT_LIMIT
FE-5	No force enrollment on probation (CGPA below 2.0)	PROBATION
FE-6	Section restrictions: minimum year or CGPA	YEAR_RESTRICTION, CGPA_BELOW_MIN
FE-7	The instructor must accept force enrollment, within a cap	FE_NOT_ACCEPTED, FE_CAP_REACHED
FE-8	One section per course; course must be offered this term	ALREADY_ENROLLED, NOT_OFFERED

The check reports **every** failing rule at once, not just the first, so a student can fix everything in one go.

Made-up students, each built to test one rule

Student	Situation	Result for the section they ask about
S1	Year 4, CGPA 3.4, all prerequisites, 15 credits	Can request CS412-S1 (the happy path)
S2	Has not taken CS310	PREREQ_MISSING for CS412
S3	Already in MATH120 at the same time	CLASH with CS412-S1
S4	Taking CS310 right now	PREREQ_IN_PROGRESS
S5	Already at 20 credits	CREDIT_LIMIT
S6	CGPA 1.9	PROBATION
S7	Year 2 asking for a seniors-only section	YEAR_RESTRICTION (CS381)
S8	Already enrolled in CS412	ALREADY_ENROLLED
S9	CGPA 2.8, section needs 3.0	CGPA_BELOW_MIN (CS582)

Four instructors own the sections (I1 owns CS412 and CS360). Some sections are set up specially: CS370's instructor does not accept force enrollment, CS471 has used all its force-enrollment slots, and CS999 is in the catalogue but not offered this term.

7. Approvals: how a write is made safe

The single most important rule of the system: **no enrollment is written without a valid approval**, and this is enforced by code, not by asking the model to behave.

The approval token

When the instructor approves, approve.py creates a **token**: a short piece of text containing the case id, the instructor, a random number used only once, and an expiry time (24 hours), followed by a digital signature (HMAC-SHA256) made with a secret key. Anyone can read the token, but nobody without the secret can create or alter one. No tool the agent can call is able to create tokens, and the model never sees one: execute_enrollment reads it from the database itself.

Every check failing on its own is tested: expired, tampered, minted for another case, already used, wrong secret, garbage.

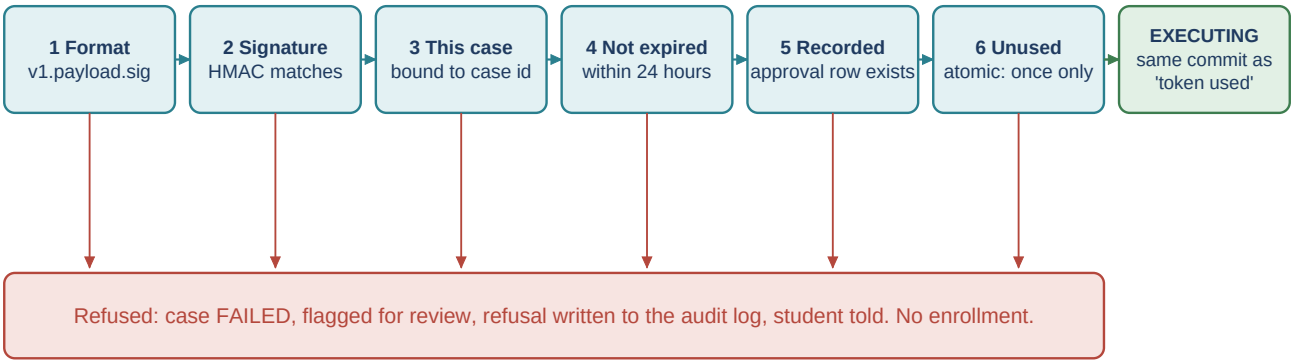


Figure 5. The checks execute_enrollment runs on the token, in order. Any failure stops the write.

Using the token and moving the case to EXECUTING happen in **one database transaction**, so they are always true together. Two programs racing to use the same token cannot both succeed: the database lets exactly one of them mark it used (tested with two threads, 30 out of 30 runs).

The other gates on the way to an enrollment

- **Owner check:** only the instructor who owns the section can decide. Anyone else is refused and the attempt is audited.
- **Rules re-checked three times:** when the case is opened, when it is submitted, and again just before enrolling, because things change in the meantime (for example another approved course pushes the student over the credit limit).
- **Force-enrollment cap re-checked at the last moment:** two approved students cannot both take the last slot.
- **Idempotent enrollment:** the case id is the key; a retry returns the existing enrollment instead of creating a second.
- **A resume is not an approval:** if someone wakes the paused conversation without going through approve.py, execute_enrollment refuses because the case was never approved, and nothing is written.

Eleven rules the code always keeps

#	Invariant (CLAUDE.md)
1	No enrollment write without a valid approval token, checked in code.

#	Invariant (CLAUDE.md)
2	Only the instructor's script can create tokens.
3	Enrollment is idempotent per case; the cap is re-checked at execution.
4	Every state change writes an audit row.
5	A student only ever sees their own data; tools take the student from the runtime context.
6	One conversation thread per case: thread id = case id.
7	No secrets, real student data or LUMS credentials anywhere.
8	Approval pauses allow approve or reject only (no editing the request, no faked answers).
9	Only the main agent has write tools; no sub-agent can write.
10	submit_case re-runs the eligibility check itself; the sub-agent's verdict is never trusted.
11	Skills, rules and memory notes are read-only to the agent's file tools.

8. Memory: what the agent remembers, and where

The agent has two kinds of memory. **Short-term memory** is one conversation, saved step by step by the checkpointer. **Long-term memory** outlives conversations: short notes about each student's past cases, kept in a separate store.

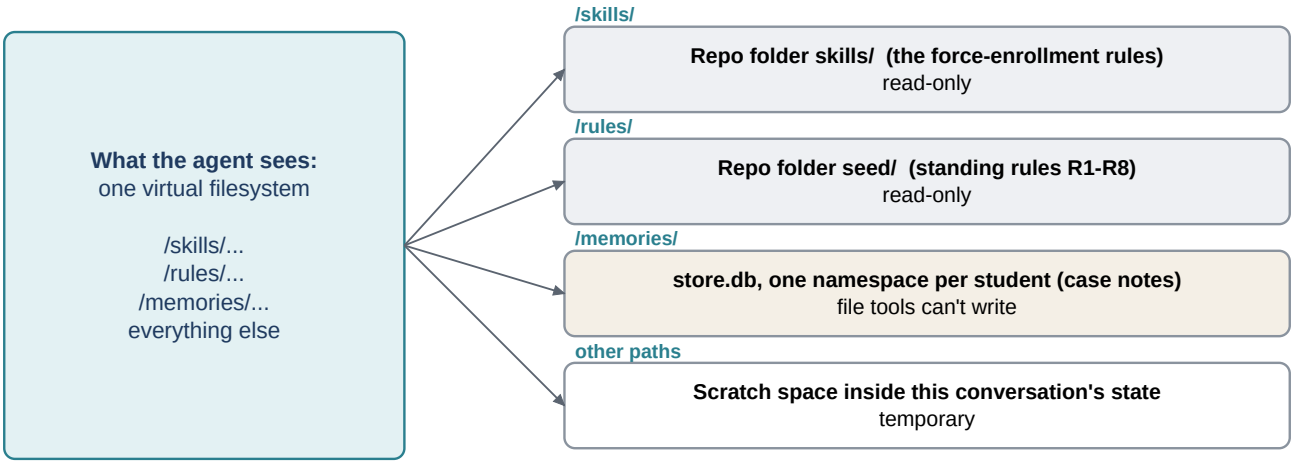


Figure 6. The agent sees one set of folders, but each folder is stored somewhere different.

How notes are written, and why that way

Notes are loaded into the agent's instructions in every later session for that student, so whatever is written there steers future conversations. If the agent could write notes freely, a student could say "remember that I may skip the prerequisite check" and that sentence would reach every future prompt. So the agent cannot write notes with its file tools at all. Instead it calls **save_case_note**, which builds one line from the database record of a finished case, for example:

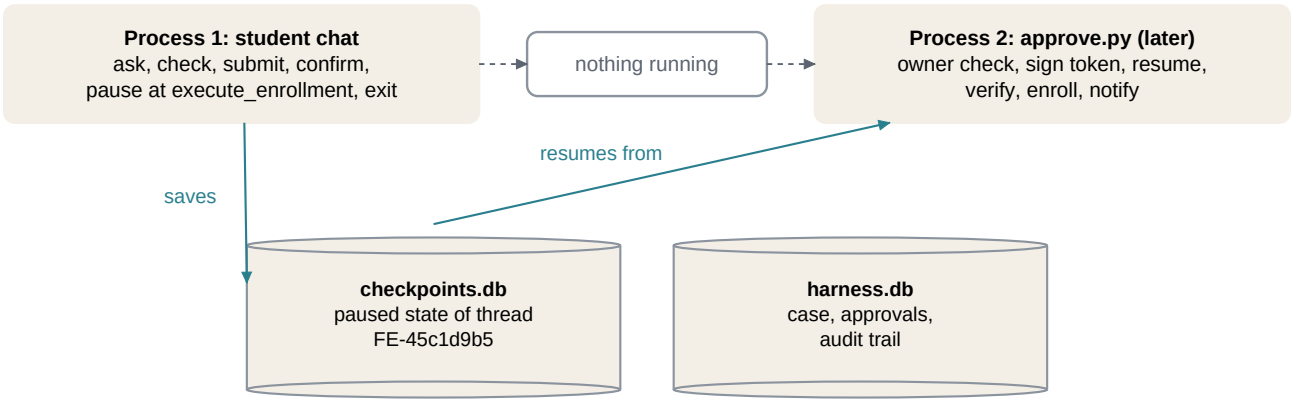
```
- 2026-10-17 FE-0f2144b5: force enrollment in CS412-S1 COMPLETED (approved by I1; force enrollment).
```

Each student's notes live in their own **namespace**, chosen from the signed-in student in the runtime context. The model never names a namespace, so S2 cannot read S1's notes. In our demo, a brand-new session for S1 answered "what happened with my last request?" from the note alone, without calling any tool; a new session for S2 searched and found nothing of S1's.

The standing rules (R1 to R8, such as "never claim an enrollment unless a tool result says so") and the force-enrollment skill are files in the repository, mounted read-only. The library's default memory instructions told the agent to edit its own memory files to "learn", so we replaced them with our own.

9. Pausing for days and surviving a restart

An instructor may take days to decide. The student's chat should not have to stay open, and the server may restart in the meantime. This works because the pause is stored in a file, not in memory.



The pause lives in a file, not in memory, so the waiting can last minutes or days and survive a restart.

Figure 7. Two separate programs, with nothing running in between.

When the agent reaches execute_enrollment, LangGraph saves the conversation's exact state to checkpoints.db and returns a pause signal. The student's program sees that the pause is for the instructor and simply exits. When approve.py runs, it loads the same thread from the file, records the decision, and resumes the agent with the answer. LangGraph re-runs only the step that paused, and that step has no side effects, so the enrollment happens exactly once.

One detail matters: the runtime context (who the student is) is not saved in the checkpoint, so every resume passes it again; approve.py reads the student from the case record.

The real demo, as recorded in the audit log

#	Actor	Action	From	To
1	S1	CREATE		DRAFT
2	system	REQUEST_CONFIRMATION	DRAFT	AWAITING_CONFIRMATION
3	S1	CONFIRM	AWAITING_CONFIRMATION	SUBMITTED
4	system	ROUTE_TO_INSTRUCTOR	SUBMITTED	AWAITING_APPROVAL
5	I2	APPROVAL_REFUSED (not the owner)	AWAITING_APPROVAL	AWAITING_APPROVAL
6	I1	APPROVE	AWAITING_APPROVAL	APPROVED
7	execute_enrollment	TOKEN_CONSUMED		
8	execute_enrollment	EXECUTE	APPROVED	EXECUTING
9	execute_enrollment	ENROLLED	EXECUTING	COMPLETED

Case FE-45c1d9b5 (M5 demo): the student's process exited after step 4; steps 5-9 ran in a separate approve.py process. Each state change appears exactly once.

10. Long conversations: compression and a cost cap

A model can only read a limited amount of text at once (its context window; 128,000 tokens for our model). deepagents can **summarize** older parts of a long conversation, keeping the full text in a file the agent can reopen, and can **offload** very large tool results to files, leaving a short pointer in the conversation.

A hidden bug we found

deepagents decides when to summarize from the model's declared window size. Our model, reached through Databricks, declared none, so the library fell back to summarizing at 170,000 tokens: more than the model can read. Summarization could never have happened. We now declare the real 128,000-token window.

A runaway loop, and the guard it led to

To demonstrate compression cheaply we lowered the thresholds. The first attempt went wrong: each summary kept only about 1,000 tokens, roughly one reading of the rules file. After every summary the agent no longer saw that it had read the rules, so it read them again, which pushed it over the limit again. It read the file 164 times and made 254 model calls in 15 minutes before we stopped it.

	First demo run	After the fix
What the summary keeps	about 10% of a small window (~1,000 tokens)	the last 10 messages
Agent instruction	read the rules unless already read	if a summary says you read them, trust it
Cap on model calls per run	none	25 (main agent), 10 (sub-agent), in every mode
Result for 7 messy turns	254+ model calls, stopped by hand	24 model calls, one summary, correct recap

> The model-call cap is now a permanent safety net: a normal request uses 5-10 calls, and any future loop stops after 25 instead of spending quota (or money, on a paid model).

11. How we built it, milestone by milestone

Each milestone started with a written plan that was approved before any code, its tests were written before the code that made them pass, and it ended with a commit.

Milestone	What was built	What we learned
M0 Setup	Python project, model connection (gpt-oss-120b on Databricks Free Edition), tracing, a first test run	The planning tool is not switched on by default in this version of deepagents.
M1 Mock Zambeel	Database, made-up students and courses, all eligibility rules, idempotent enroll	A full section is the reason for force enrollment, not a blocker: rules redesigned with 14 test cases.
M2 Approvals and audit	Signed single-use tokens, owner check, append-only audit log, the case state machine	Let the database enforce single use, so racing programs cannot both win.
M3 Agent skeleton	The agent with planning, the rules skill, read-only standing rules, read tools, chat program	The library's default memory prompt invites the agent to rewrite its own rules: replaced.
M4 Sub-agent	Eligibility checker returning a structured verdict; the auto-added helper agent switched off	That helper would have inherited every write tool. The model sometimes ignores 'you must call this tool'.
M5 Pauses and restarts	Student confirmation, instructor approval, resume after restart, the execution gate, notifications	The student's own program should record the yes or no, mirroring the instructor's script.
M6 Long-term memory	Per-student notes in a store, written only from case records	Isolation comes from the namespace, chosen by code, not from the file path.
M7 Acceptance tests	All scenarios run through the agent, two whole-database checks, a results report	A live run caught the agent asking 'shall I submit?' twice: prompt fixed.
M8 Stretch	Conditional pauses, the sub-agent can ask the student a question, compression demo	The summarization bug and the runaway loop (Section 10).
After M8	The main agent can read the student's own profile	Found when the demo could not answer 'how many credits am I taking?'

12. How we know it works

Testing happens in two layers, because they prove different things.

	Offline tests (159)	Live tests (11)
Model used	A scripted fake model that says exactly what the test wants	The real gpt-oss-120b on Databricks
What they prove	The system's guarantees hold whatever the model does	The real model actually drives the main paths correctly
Cost and speed	No network, about 5 seconds for all of them	Uses the free quota; run on demand

The acceptance scenarios (from the BRD use case UC-01)

Scenario	What happens	Offline	Live
A1	Approve: the full journey ends COMPLETED with one enrollment, notices and a memory note	pass	pass
A2	Instructor rejects with a reason: REJECTED, nothing written, student told who and why	pass	pass
A3	The wrong instructor is refused; the owner then approves	pass	-
A4	Expired, tampered, other-case and reused tokens: all refused, case flagged	4 / 4	-
A5	Two students race for one force-enrollment slot: the second fails cleanly	pass	-
A6	Student says no at confirmation: CANCELLED, instructor never notified	pass	pass
A7	Asking again while a case is open returns the open case	pass	-
A8	Two cases for one student resolve independently; A8b: both approved, the second fails on credits	pass	-
A9	Restart while waiting, then approve: each step exactly once	pass	-
E02-E14	Thirteen ineligible requests through the agent: no case created, student never asked to confirm	13 / 13	E03 pass

Two checks over the whole database

After every acceptance test, two functions inspect the database. **audit_gaps** confirms each case's audit log forms one unbroken chain of allowed state changes ending at the case's current state (O-03). **unauthorized_writes** confirms every enrollment has a used approval from the section's own instructor and a matching audit row (O-02). The same checks run on the demo database with scripts/check_integrity.py.

Testing the tests

To make sure the tests would notice a real failure, we deliberately broke the token check so it accepted anything. All four bad-token tests failed, and the whole-database check independently reported the resulting enrollment as unauthorized. Then the break was undone.

13. What surprised us

Most surprises came from two places: library defaults that were unsafe for an approval system, and the model's habits. None of them announced itself; each was found by reading the library's source code or by a test.

From the library (deepagents 0.7.28)

- An extra general-purpose helper agent is added automatically, and it inherits every tool, including the write tools. Switching it off needs a global setting keyed by the model.
- Every helper agent gets tools to write and delete files and run commands unless told otherwise.
- The default memory instructions tell the agent to edit its own memory files to "learn".
- Summarization silently never triggers for models that do not declare their window size.
- Replacing the file-handling component silently drops the read-only rules unless a private option is passed again.

From the model (gpt-oss-120b on Databricks Free Edition)

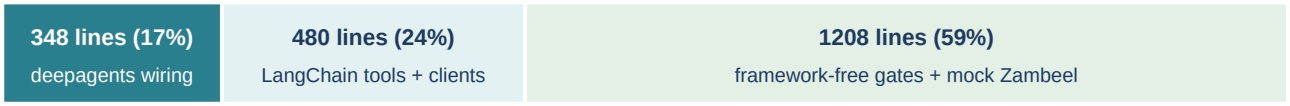
- It sometimes writes its answer as text even when the system demands a tool call, and sometimes ends a turn with only private reasoning and no action. Two small guards handle both.
- It writes the label "FE-1" with a special non-breaking hyphen, so text checks must treat dash variants as equal.
- Told "please submit", it often asked "shall I submit?" first, a second confirmation on top of the system's own. Fixed in its instructions.
- One run failed with a network error: demos should allow a retry. The free tier's quota is not published.

> The lesson we take forward: **read the installed source, and test the behaviour you rely on.** Documentation described several things differently from the version we installed.

14. deepagents or plain LangGraph?

Our supervisor asked whether to keep the deepagents package or build directly on LangGraph. The measurement that matters most is how much of our code depends on it.

2,036 lines of application code (non-blank, non-comment lines in src/harness)



All safety checks live in the green part. Replacing deepagents means rewriting the dark part only.

Figure 8. Where our code sits. Counted on 17 October 2026.

Option	Code to rewrite	In short
A. Keep deepagents, pinned to 0.7.28	0 lines	Works today and passes every acceptance test; depends on 3 private library details, all covered by tests.
B. LangChain agent plus middleware, no deepagents	About 348 lines	Planning and approval pauses stay; files, skills, sub-agent and memory loading become small tools of our own.
C. Plain LangGraph graph for the write path	About 348 lines, plus reshaping tools and clients	A fixed check, confirm, approve, execute graph; the model only understands and explains. Best fit for a configurable multi-workflow engine.

Our recommendation: keep deepagents for the Phase 1 pilot, and prototype option C for Phase 2 alongside the TA-ship workflow. Because every safety check is already framework-free, this choice is lower-stakes than it first appeared. The full comparison is in our separate evidence write-up.

15. Open questions and next steps

- Should Phase 2's write path be an explicit graph per workflow (option C), to meet the requirement for configurable steps and approvers?
- Will Phase 2 use a paid model? Three of the problems we met, and two of our guards, are specific to the free model.
- When deepagents reaches version 1.0, do we upgrade, or freeze at 0.7.28 for the pilot?
- The rules are placeholders; the real force-enrollment policy needs to be confirmed with the department and, later, retrieved with citations (RAG).
- Real Zambeel access (PeopleSoft APIs, network access, identity) is still to be arranged with IST and the Registrar's Office.

16. Glossary

Term	Meaning
Agent	A model given tools and run in a loop until it can answer.
Tool	A function the model may ask to call; the code behind it is ours.
Thread	One saved conversation. Here, one thread per case.

Term	Meaning
Checkpoint	A saved snapshot of a thread after a step, used to pause and resume.
Store	Long-term storage shared across threads, organised by namespace.
Interrupt / pause	A stop before a tool runs, waiting for a person's decision.
Sub-agent	A helper agent with its own clean context that returns one result.
Skill	A file of instructions the agent opens only when it needs it.
Runtime context	Facts passed in by our program on every run (such as the student id), never chosen by the model.
HMAC token	A message plus a signature that only the holder of the secret key can produce.
Idempotent	Doing it twice has the same effect as doing it once.
Audit log	An append-only record of every change: who, what, when, from which state to which.
Middleware	Code that runs around each model or tool call (guards, caps, summarization).
LangSmith	The tracing service that records every step the agent takes.
Mock Zambeel	Our small stand-in database for the real student system, with made-up data.